



LO17 : RAGoût indexation et recherche de recettes

Rapport de Projet RAG

Membre 1 :
Delaby Thibault (25%)

Membre 2 :
Pilache Ellen (25%)

Membre 3 :
Ribout Charlotte (25%)

Membre 4 :
Sideri Christelle (25%)

Groupe :
Groupe D1-G1 LO17



Sommaire

1	RAG naïf	3
1.1	Jeu de données	3
1.2	Fonctionnement général du RAG	3
2	Améliorations	4
2.1	Température	4
2.2	Changement de modèle	4
2.3	Prompt Système	4
2.4	Filtrage du Retriever	5
2.5	CRAG léger	5
2.6	Mémoire	5
2.7	Query Construction	5
3	Evaluation et Interface	6
3.1	Évaluation du RAG amélioré	6
3.2	Streamlit	7
3.3	Déploiement	7
A	Récapitulatif des décisions d'implémentation	9



Introduction

Ce projet a pour objectif la conception et l'évaluation d'un système de Génération Augmentée par la Recherche, plus couramment appelé RAG pour *Retrieval-Augmented Generation*. Un système RAG combine deux approches complémentaires : d'une part, un module de recherche d'information, chargé d'identifier dans une base documentaire les contenus les plus pertinents par rapport à une requête utilisateur ; d'autre part, un modèle de génération de texte, qui exploite ces informations pour produire une réponse contextualisée. Contrairement à un modèle de langage utilisé seul, le RAG permet donc d'ancrer la génération dans des documents externes, ce qui améliore la précision des réponses et limite en partie le risque d'hallucinations.

Nous avons choisi d'appliquer cette approche au domaine culinaire, en nous appuyant sur un corpus de recettes extraites du site web 750g. Ce choix résulte d'une volonté de notre groupe de travailler sur un cas d'usage à la fois concret, accessible et suffisamment riche pour évaluer les différentes composantes d'un système RAG. Les recettes de cuisine présentent en effet plusieurs avantages : elles possèdent une structure relativement régulière, avec des éléments facilement identifiables tels que le titre, la liste des ingrédients, les quantités, les étapes de préparation, le temps de cuisson ou encore le nombre de portions. Cette organisation facilite la constitution d'une base documentaire exploitable et permet d'étudier plus finement la qualité de la recherche d'information.

Le site 750g constitue également une source pertinente pour ce projet, car il propose un grand nombre de recettes variées et accessibles publiquement. Sa structure web permet l'extraction automatisée des données nécessaires à la construction du corpus, ce qui rend possible la création d'une base suffisamment large pour tester différentes stratégies de recherche et de génération. Le domaine culinaire se prête par ailleurs bien à l'évaluation qualitative des réponses produites : il est possible de vérifier si les ingrédients sont cohérents, si les étapes sont respectées ou encore si les recommandations générées restent plausibles.

Dans un premier temps, nous avons implémenté une architecture de RAG naïf afin d'obtenir une première version fonctionnelle du système. Puis, dans un second temps, nous proposons diverses optimisations visant à accroître les performances et la fiabilité du système, notamment par l'enrichissement de la base documentaire, le perfectionnement du module d'extraction d'information (retriever) et la réduction des hallucinations. Enfin, une interface utilisateur interactive sous Streamlit a été développée et déployée afin de rendre le système facilement utilisable.



Partie 1

RAG naïf

Ce premier prototype repose sur une architecture RAG classique (Retrieval Augmented Generation). L'objectif est de permettre à un utilisateur d'obtenir des recommandations de recettes de cuisine à partir d'une base documentaire composée de recettes indexées. Lorsqu'une question est posée, le système récupère les documents les plus pertinents avant de générer une réponse à l'aide d'un modèle de langage.

1.1 Jeu de données

Le jeu de données utilisé est constitué d'un ensemble de recettes de cuisine provenant du site 750g. Chaque recette est ensuite stockée au format JSON avec plusieurs informations utiles : le titre, la difficulté, le temps de préparation, le temps de cuisson, le nombre de personnes ou de pièces, le matériel nécessaire, les ingrédients, ainsi que les instructions de préparation.

Pour construire cette base de données, nous avons utilisé deux scripts Python. Le premier, `crawler_url_recettes.py`, permet de parcourir les pages de catégories du site 750g afin d'en extraire automatiquement les liens des recettes. Il récupère les URLs présentes sur les pages, gère la pagination et évite les doublons grâce à un ensemble `set`. Le résultat est sauvegardé dans un fichier `recettes_links.json`, qui contient toutes les URLs à exploiter.

Le second script, `crawler_recettes.py`, utilise ensuite ce fichier d'URLs pour accéder à chaque page de recette et en extraire les informations détaillées. Des fonctions de nettoyage sont également utilisées afin de supprimer les éléments parasites présents dans les pages HTML, comme certains boutons ou textes inutiles.

À la fin de cette étape, les recettes sont sauvegardées dans le fichier `recettes_details.json`. Ce fichier constitue notre base documentaire principale. Avant l'indexation, les recettes sont chargées puis découpées en documents exploitables par le système RAG. Ces documents sont ensuite transformés en représentations vectorielles grâce au modèle `gemini-embedding-001`, puis stockés dans une base vectorielle Chroma.

1.2 Fonctionnement général du RAG

Le fonctionnement du système suit les étapes suivantes :

1. L'utilisateur saisit une question ou une liste d'ingrédients.
2. La question est transformée en vecteur grâce au modèle d'embedding.
3. Le retriever recherche les documents les plus proches dans la base Chroma.
4. Les documents récupérés sont fournis au modèle Gemini sous forme de contexte.
5. Le modèle génère une réponse en s'appuyant sur les recettes retrouvées.

1.2.1 Description des paramètres du RAG naïf

Nous avons implémenté notre RAG naïf en nous basant sur le notebook *Gemini API : Question Answering using LangChain and Chroma* et en utilisant le package LangChain pour la connexion au modèle de langage. Le retriever effectue une recherche par similarité et sélectionne les 6 documents les plus pertinents. Le générateur utilise Gemini 2.5 Flash avec une température de 0.7 pour une certaine créativité. Enfin, nous avons conservé le prompt système d'origine afin de maintenir une approche naïve.



Partie 2

Améliorations

Dans cette section, nous nous concentrerons sur les améliorations implémentées à notre RAG. La liste complète des améliorations considérées est disponible en section A.

2.1 Température

La température est un paramètre de génération du LLM qui contrôle le degré d'aléatoire dans les réponses produites. Une valeur proche de 0 rend le modèle déterministe : il favorise systématiquement le token le plus probable, ce qui produit des réponses précises et factuelles. À l'inverse, une valeur élevée (proche de 1 ou au-delà) introduit plus de créativité et de variété, au risque d'obtenir des réponses moins fiables.

Dans le cadre de notre RAG culinaire, l'objectif est de fournir des recettes fidèles à la base de données sans invention ni hallucination. Nous avons donc opté pour une température basse :

```
temperature = 0.1
```

Cette valeur garantit des réponses cohérentes et reproductibles tout en conservant une légère flexibilité dans la formulation des réponses, évitant ainsi un rendu trop mécanique.

2.2 Changement de modèle

Le modèle utilisé au sein du générateur a été changé pour Gemini 3.1 Flash Lite afin de contourner les restrictions de tokens et de requêtes du Free-Tier Google et d'augmenter la rapidité de réponse du RAG. Bien que plus performant et rapide, ce modèle conserve une qualité équivalente à Gemini 2.5 Flash.

2.3 Prompt Système

Le prompt système est une instruction transmise au LLM avant toute interaction utilisateur. Il définit le rôle, les règles de comportement et le format de réponse attendu. C'est un levier essentiel pour cadrer le modèle et prévenir les hallucinations.

Notre prompt système définit les éléments suivants :

- **Identité** : le modèle incarne RAGoût, un chef cuisinier expert et assistant culinaire.
- **Règles absolues** : le modèle ne propose que des recettes issues du contexte récupéré ; il ne modifie jamais les ingrédients ni les étapes d'une recette, sauf pour adapter les quantités proportionnellement au nombre de personnes demandé ; si aucune recette ne correspond, il le dit explicitement sans inventer d'alternative.
- **Gestion des ingrédients incomplets** : si l'utilisateur fournit une liste d'ingrédients sans préciser qu'il peut faire des courses, le modèle propose la recette la plus adaptée en précisant les ingrédients manquants, sans refuser de répondre.
- **Gestion des quantités** : si un nombre de personnes est précisé, les quantités sont adaptées proportionnellement avec des arrondis réalistes (jamais 0,5 œuf, jamais 0,3 cuillère à soupe).
- **Format de réponse structuré** : nom de la recette, temps de préparation, ingrédients disponibles, ingrédients à prévoir, et une justification du choix de la recette.

Ce prompt complet permet également de limiter les mauvaises utilisations ou les demandes qui sortent du cadre d'un RAG culinaire.



2.4 Filtrage du Retriever

Par défaut, Chroma retourne les k documents les plus proches d'une requête selon une similarité vectorielle. Nous avons expérimenté un filtrage par seuil relatif afin d'éliminer les documents dont le score était trop éloigné du meilleur résultat obtenu.

$$\text{score}(d) \geq \text{score}_{\max} - \Delta$$

avec $\Delta = \text{SEUIL_RELATIF} = 0,05$.

Cette approche permet de réduire une partie du bruit transmis au LLM. Cependant, nous avons observé que les scores de similarité variaient fortement selon la formulation de la requête. De plus, des documents peu pertinents pouvaient obtenir des scores proches des meilleurs résultats, ce qui limitait l'efficacité du filtrage.

Nous avons donc considéré cette approche comme une amélioration intermédiaire plutôt qu'une solution définitive.

2.5 CRAG léger

Afin de réduire davantage les faux positifs du retrieval vectoriel, nous avons implémenté une approche inspirée de CRAG (Corrective Retrieval-Augmented Generation).

Le principe consiste à récupérer plusieurs documents candidats avec Chroma, puis à demander au LLM d'évaluer leur pertinence par rapport à la question utilisateur. Seuls les documents jugés réellement utiles sont conservés pour la génération de la réponse.

Cette étape supplémentaire permet de corriger certaines erreurs du retrieval vectoriel, notamment lorsque plusieurs recettes sont proches sémantiquement mais ne répondent pas exactement à la demande formulée.

2.6 Mémoire

Dans un RAG classique, chaque question est traitée de manière indépendante : le modèle n'a aucune connaissance des échanges précédents. Cela pose problème dès qu'un utilisateur formule une question de suivi comme « *Et sans les tomates ?* », qui est incompréhensible sans le contexte de la question précédente.

Pour pallier cela, nous avons implémenté un système de mémoire conversationnelle reposant sur deux mécanismes :

- Historique de conversation : chaque échange (question utilisateur + réponse du modèle) est stocké sous forme de messages `HumanMessage` et `AIMessage` dans une liste `chat_history`, transmise à chaque nouvelle requête.
- Reformulation autonome de la question : avant le retrieval, si un historique existe, la question est reformulée par le LLM en une question autonome et complète. Ainsi, « *Et sans les tomates ?* » devient « *Quelles recettes de pâtes peut-on faire en 20 minutes sans tomates ?* », ce qui permet un retrieval pertinent.

Ce système de double prompt : un prompt de reformulation suivi du prompt de réponse garantit que le retrieval reste efficace tout au long d'une conversation sans alourdir inutilement le contexte transmis au LLM.

2.7 Query Construction

La query construction consiste à demander au LLM d'extraire des filtres structurés depuis la question de l'utilisateur (temps de préparation, type de plat, régime alimentaire, etc.) afin de restreindre le retrieval en amont. Cette technique était particulièrement pertinente pour notre cas, nos documents étant bien structurés.

Cependant, nous avons rapidement atteint les limites de tokens imposées par l'API Gemini. Le contexte transmis au LLM, historique de conversation, documents récupérés et prompt système, étant déjà conséquent, l'ajout d'une étape de construction de filtres aurait aggravé ce problème. Nous avons donc renoncé à cette amélioration.



Partie 3

Evaluation et Interface

La dernière étape de notre projet a consisté à évaluer notre RAG puis concevoir à une interface graphique pour notre système RAG et enfin à le déployer afin de le rendre facilement accessible aux utilisateurs.

3.1 Évaluation du RAG amélioré

3.1.1 Méthodologie d'évaluation

L'évaluation du système RAG amélioré repose sur deux approches complémentaires.

La première approche s'inspire du framework RAGAS, dont nous avons réimplémenté manuellement les métriques en raison d'incompatibilités techniques avec notre environnement Google Colab. Notre implémentation utilise Gemini comme juge externe (*LLM-as-a-judge*) : pour chaque question de test, le modèle reçoit simultanément la question, la réponse générée et les contextes récupérés, puis attribue un score entre 0 et 1 sur quatre métriques :

- **Faithfulness** : fidélité de la réponse aux documents récupérés
- **Answer Relevancy** : pertinence de la réponse par rapport à la question
- **Context Precision** : qualité des documents récupérés par le retriever
- **Noise Sensitivity** : résistance du modèle aux documents non pertinents

La seconde approche consiste en une vérification manuelle : pour un sous-ensemble de questions, nous avons consulté directement le fichier JSON de recettes afin de vérifier si les documents récupérés par le RAG correspondaient effectivement aux meilleures réponses attendues, et si les informations fournies par le système étaient factuellement exactes. Cette méthode permet de détecter les cas où le juge LLM lui-même pourrait se tromper dans son évaluation. Mais cette approche manuelle est assez chronophage.

3.1.2 Améliorations évaluées

Par rapport au RAG naïf, le système évalué intègre les améliorations suivantes :

- **Prompt enrichi** : instructions détaillées sur la gestion des ingrédients, des contraintes de temps, du nombre de personnes et des adaptations de quantités
- **Température réduite** (0.1) : réponses plus déterministes et reproductibles
- **Filtrage par seuil relatif** : les documents récupérés par Chroma sont filtrés avec un seuil relatif de 0.05 par rapport au meilleur score de similarité.

3.1.3 Résultats

Le tableau 3.1 présente les scores obtenus sur la question « *J'ai faim un oignon et 20 min* », représentative des requêtes combinant contrainte de temps et type d'ingrédient.

Métrique	RAG naïf "LLM as a Judge"	RAG amélioré "LLM as a Judge"	RAG amélioré "A la main"
Faithfulness	1.00	1.00	1.00
Answer Relevancy	1.00	1.00	0.50
Context Precision	0.50	0.80	0.33
Noise Sensitivity	0.10	0.00	0.00

TABLE 3.1 – Scores d'évaluation du RAG naïf "LLM as a Judge" vs RAG amélioré "LLM as a Judge" vs RAG amélioré "A la main"

Cette question illustre bien l'apport de nos améliorations sur des requêtes à contraintes multiples : ingrédient unique, limite de temps et adaptation des quantités. La progression la plus notable concerne la précision du contexte, qui passe de 0.50 à 0.80 selon le juge automatique grâce au filtrage par seuil relatif et au prompt enrichi. Le seuil écarte les documents peu pertinents avant transmission au LLM, tandis



que le prompt guide explicitement le modèle dans la sélection de la recette la mieux adaptée. Néanmoins, l'évaluation manuelle nuance fortement ce résultat avec une *Context Precision* qui retombe à 0.33, ce qui démontre qu'un évaluateur humain se montre beaucoup plus exigeant sur la stricte pertinence et l'utilité des blocs de contexte sélectionnés pour répondre à la requête.

La fidélité atteint 1.0 dans l'ensemble des configurations (automatiques comme manuelle), ce qui s'explique par la qualité structurale de notre corpus : les documents issus du scraping de 750g.com présentent un format homogène avec titre, liste d'ingrédients et instructions claires, permettant même au RAG naïf de produire des réponses fidèles aux sources. L'évaluation "A la main" valide d'ailleurs totalement ce score, confirmant l'absence d'hallucination ou de dérive par rapport aux données d'origine.

Pour des questions plus simples sans contraintes combinées, les différentes versions obtiennent des scores comparables sur toutes les métriques. Le gain apporté par nos améliorations est donc particulièrement visible sur les requêtes complexes, là où le prompt naïf manque d'instructions pour gérer simultanément plusieurs critères. C'est pourtant sur ces requêtes complexes que se révèle la limite de l'approche *LLM as a Judge* : l'*Answer Relevancy* s'effondre à 0.50 lors de l'évaluation manuelle, contre 1.00 pour le LLM. Cette divergence majeure s'explique par la formulation implicite « *J'ai faim* » : là où le LLM évaluateur valide mécaniquement la simple présence de l'ingrédient et le respect du temps, l'humain pénalise le manque de réalisme ou l'incohérence culinaire de la recette finale si elle s'avère incomplète ou peu praticable au quotidien.

3.2 Streamlit

Afin de rendre notre assistant culinaire utilisable par un public non technique, nous avons développé une interface graphique à l'aide de la bibliothèque Streamlit. Ce framework Python permet de créer rapidement des applications web interactives sans avoir recours aux technologies classiques du développement web telles que HTML, CSS ou JavaScript.

L'interface adopte un format conversationnel similaire à celui d'un assistant virtuel. L'utilisateur peut poser librement une question concernant une recette, des ingrédients ou des contraintes de préparation.

L'historique des échanges est conservé au cours de la session grâce au mécanisme `st.session_state`. Cette fonctionnalité permet de maintenir le contexte de la conversation et d'utiliser notre système de reformulation des questions lorsque l'utilisateur fait référence à un échange précédent.

Afin d'améliorer les performances de l'application, le modèle Gemini ainsi que la base vectorielle Chroma sont chargés une seule fois au démarrage grâce au mécanisme de cache de Streamlit. Cette mise en cache évite de reconstruire l'ensemble de la chaîne RAG à chaque interaction et réduit significativement le temps de réponse.

Enfin, l'interface offre la possibilité de consulter les extraits de recettes récupérés lors de la phase de recherche documentaire. Ces informations sont affichées dans un panneau déroulant, permettant à l'utilisateur de visualiser les documents ayant servi à générer la réponse. Cette fonctionnalité apporte davantage de transparence sur le fonctionnement du système.

3.3 Déploiement

Une fois l'application finalisée, celle-ci a été déployée sur la plateforme Streamlit Community Cloud. Cette solution permet d'héberger gratuitement une application Streamlit directement à partir d'un dépôt GitHub et de la rendre accessible via un simple navigateur web.

Nous avons choisi un déploiement public afin de permettre à toute personne de tester notre assistant culinaire. Toutefois, l'application repose sur l'API Gemini de Google, dont la version gratuite impose des quotas d'utilisation (15 requêtes par minutes maximum). Le nombre de requêtes pouvant être effectuées est donc limité, ce qui peut occasionnellement rendre le service indisponible lorsque ces quotas sont atteints.

L'application est accessible à l'adresse suivante : <https://duflan.streamlit.app/>



Conclusion

Résumé du travail

Ce projet a abouti à la création de **RAGoût**, un système de recommandation de recettes fondé sur une architecture RAG. À partir d'un corpus extrait du site 750g, nous avons structuré et indexé les données culinaires (ingrédients, étapes, matériel) dans une base vectorielle Chroma à l'aide d'embeddings `gemini-embedding-001`. Une première version naïve a permis d'évaluer le système selon plusieurs métriques (*faithfulness*, *answer relevancy*, etc.) avant d'optimiser la génération des réponses via le réglages du prompt, des paramètres et la mise en place d'un contexte. Enfin, le projet a été concrétisé par le développement et le déploiement public d'une interface graphique sur Streamlit Community Cloud.

Bilan

Le bilan est positif : **RAGoût** répond efficacement à des requêtes contextualisées par des ingrédients disponibles ou des contraintes matérielles, validant ainsi la pertinence du domaine culinaire pour ce type d'application. Néanmoins, des limites subsistent. La pertinence des réponses reste fortement liée de la précision du *retriever*, le risque d'hallucination n'est pas totalement nul et la base documentaire se cantonne aux données de 750g. De plus, la version déployée est contrainte par les quotas de requêtes gratuites de l'API Gemini.

Contributions et perspectives

Grâce à une répartition équilibrée des tâches, allant de la collecte des données à l'évaluation puis au déploiement, chaque membre de l'équipe a pu participer aux différentes étapes du projet et mieux comprendre son fonctionnement global. Cette expérience a également montré à quel point la qualité des prompts et le réglage du retriever influencent les performances du système. Pour améliorer encore la solution, il serait intéressant d'intégrer de nouvelles sources d'information, notamment via des recherches web en temps réel.



Annexe A — Récapitulatif des décisions d'implémentation

Technique	Justification	Décision/Justification
Query Construction	Permet de filtrer par critères structurés (temps, type de plat). Parfaitement adapté à nos documents structurés	Implémentation pertinente mais nous n'avions pas assez de token pour le faire
Multi-Query	Technique de base robuste qui élargit le retrieval en reformulant la requête sous plusieurs angles	Améliore le retrieval sauf que le notre est bon grâce à la qualité de nos données
RAG Fusion	Re-rank naturel des documents issus du Multi-Query, améliore directement la pertinence	même chose
HyDE	Génère un document hypothétique pour un retrieval sémantique plus précis, approche innovante	même chose
Sémantique Routing	Meilleurs résultats que le routing logique, couplé à HyDE pour maximiser la pertinence	Un seul document donc inutile
Routing Logique	Utile pour les grandes sources documentaires	Non retenu
Mémoire des prompts	Assure la cohérence conversationnelle entre les échanges	Implémenté
Prompt Engineering	Réduit les hallucinations et gère les cas d'ingrédients incomplets	Implémenté
Température	Calibrage fin selon le type de réponse : précision factuelle vs suggestion créative. On ne demande pas au RAG d'être créatif et cela permet de réduire les hallucinations	Implémenté (température faible)
CRAG	Améliore la robustesse du retrieval mais introduit une latence supplémentaire.	Implémenté une variante avec où un llm tri les documents pour ne garder que les meilleurs (évite de surcharger le llm qui répond)
RAPTOR	Utile si la volumétrie des documents augmente significativement, donc inutile pour notre cas	Non retenu
Query Translation	Le retrieval existant est déjà performant, surcoût non justifié	Non retenu
Décomposition	Questions trop simples dans notre contexte, complexité et latence accrues inutilement	Non retenu
Step Back	Cas de requêtes trop spécifiques non anticipé dans notre usage	Non retenu
Logique Routing	Une seule base de données, routing inutile	Non retenu
Adaptive RAG	Toutes les questions pertinentes nécessitent un retrieval, filtrage inutile	Non retenu
Multi Representation	Documents déjà structurés, surcoût d'indexation non justifié	Non retenu
ColBERT	Complexité d'implémentation élevée pour un gain marginal dans notre cas d'usage	Non retenu